

UNIVERSIDADE NOVE DE JULHO

CURSO DE CIÊNCIA DA COMPUTAÇÃO

OPTIFLOW LOGÍSTICA INTELIGENTE

Plataforma de Otimização de Operações Logísticas para PMEs de E-commerce

DANIEL TOMAZI DE OLIVEIRA

São Paulo — SP

2026

DANIEL TOMAZI DE OLIVEIRA

OPTIFLOW LOGÍSTICA INTELIGENTE

Plataforma de Otimização de Operações Logísticas para PMEs de E-commerce

Projeto Integrador apresentado ao Curso de Ciência da Computação da Universidade Nove de Julho (UNINOVE), como requisito parcial para aprovação nas disciplinas de Gestão de Projetos, Análise de Dados, Segurança da Informação e Pesquisa Operacional.

Orientador(a): Prof. Felipe Santos de Jesus

São Paulo — SP

2026

RESUMO

O presente trabalho consiste no desenvolvimento do Projeto Integrador OptiFlow Logística Inteligente, uma concepção técnica e acadêmica de plataforma SaaS (*Software as a Service*) para otimização de operações logísticas de última milha, voltada para pequenas e médias empresas (PMEs) de e-commerce. O projeto integra quatro disciplinas — Gestão de Projetos, Análise de Dados, Segurança da Informação e Pesquisa Operacional — em torno de um problema de negócio concreto: a ineficiência operacional na logística de entregas para PMEs brasileiras, cujo mercado movimentava R\$ 8,5 bilhões anuais. Na disciplina de Gestão de Projetos, foram elaborados oito artefatos de planejamento, incluindo Termo de Abertura (TAP), Estrutura Analítica do Projeto (EAP/WBS), Matriz RACI, Cronograma Gantt, Plano de Riscos e Backlog Ágil. Para Análise de Dados, foi construído um pipeline ETL completo em Python, com geração de dataset logístico sintético de 200 registros, cálculo de indicadores-chave de desempenho (KPIs) e produção de visualizações analíticas utilizando as bibliotecas pandas, matplotlib e seaborn. No módulo de Segurança da Informação, projetou-se uma arquitetura de autenticação multicamada com bcrypt, JWT e TOTP, realizou-se mapeamento de ameaças via STRIDE, construiu-se Matriz GUT e elaborou-se plano de adequação à LGPD (Lei nº 13.709/2018). Em Pesquisa Operacional, foram modelados e resolvidos dois problemas de Programação Linear Inteira (PLI) utilizando a biblioteca PuLP com solver CBC: otimização de rotas de entrega (variante CVRP) e alocação eficiente de motoristas por região, cada um com análise de cinco cenários operacionais distintos. Os resultados demonstram que a abordagem integrada pode reduzir custos logísticos em até 26% e elevar a taxa de entregas no prazo de 77% para 91%.

Palavras-chave: Logística. Otimização. Programação Linear Inteira. Análise de Dados. Segurança da Informação. LGPD. Gestão de Projetos.

SUMÁRIO

1 INTRODUÇÃO

1.1 Justificativa

1.2 Objetivos

1.3 Metodologia

2 DESCRIÇÃO DA EMPRESA E CONTEXTO DE NEGÓCIO

2.1 A empresa OptiFlow

2.2 Diagnóstico dos problemas de negócio

2.3 Proposta de valor

3 GESTÃO DE PROJETOS

3.1 Termo de Abertura do Projeto (TAP)

3.2 Estrutura Analítica do Projeto (EAP/WBS)

3.3 Matriz RACI

3.4 Cronograma e Backlog Ágil

3.5 Plano de Riscos

4 ANÁLISE DE DADOS

4.1 Dataset logístico simulado

4.2 Pipeline ETL

4.3 Indicadores-chave de desempenho (KPIs)

4.4 Visualizações analíticas

5 SEGURANÇA DA INFORMAÇÃO

5.1 Arquitetura de autenticação

5.2 Mapeamento de ameaças (STRIDE)

5.3 Matriz GUT

5.4 Adequação à LGPD

6 PESQUISA OPERACIONAL

6.1 Problema 1 — Otimização de rotas de entrega

6.2 Problema 2 — Alocação de motoristas por região

6.3 Análise de cenários

6.4 Implementação computacional

7 RESULTADOS E DISCUSSÃO

8 CONSIDERAÇÕES FINAIS

REFERÊNCIAS

1 INTRODUÇÃO

O comércio eletrônico brasileiro cresceu mais de 300% na última década, consolidando-se como um dos setores mais dinâmicos da economia nacional. Entretanto, esse crescimento exponencial trouxe consigo desafios operacionais significativos, especialmente no segmento de logística de última milha — a etapa final do processo de entrega, do centro de distribuição até o endereço do cliente. Enquanto grandes marketplaces investem em tecnologia proprietária de roteamento e gestão de frotas, as pequenas e médias empresas (PMEs) de e-commerce permanecem dependentes de processos manuais e ferramentas básicas, resultando em altos custos logísticos, atrasos frequentes e insatisfação dos clientes (FAYYAD; PIATETSKY-SHAPIO; SMYTH, 1996).

Nesse contexto, o presente trabalho propõe a concepção técnica e acadêmica da plataforma **OptiFlow Logística Inteligente**, uma solução SaaS de otimização logística para PMEs, desenvolvida como Projeto Integrador com abordagem multidisciplinar.

1.1 Justificativa

O mercado de logística de última milha no Brasil movimenta R\$ 8,5 bilhões anuais, porém a maior parte das soluções tecnológicas disponíveis é inacessível para PMEs, tanto pelo custo quanto pela complexidade de implantação. Dados do setor indicam que empresas sem roteirização inteligente desperdiçam entre 20% e 35% em combustível, apresentam taxa média de atraso nas entregas de 23% e possuem custo logístico que representa de 15% a 25% do valor do pedido.

A OptiFlow justifica-se pela oportunidade de aplicar, de forma integrada, conceitos de Gestão de Projetos, Análise de Dados, Segurança da Informação e Pesquisa Operacional para conceber uma solução que atenda a essa lacuna de mercado.

1.2 Objetivos

Objetivo geral: Desenvolver a concepção técnica e acadêmica da plataforma OptiFlow, integrando quatro disciplinas em torno de um problema de negócio concreto na área de logística.

Objetivos específicos:

a) Elaborar a documentação completa de gestão de projetos, incluindo TAP, EAP, RACI, Cronograma Gantt, Plano de Riscos, Plano de Comunicação, Backlog Ágil e Estimativa de Custos;

- b) Desenvolver um pipeline de análise de dados com geração de dataset sintético, limpeza, cálculo de KPIs e produção de visualizações em Python;
- c) Projetar a arquitetura de segurança da informação, com mecanismos de autenticação, mapeamento de ameaças e plano de adequação à LGPD;
- d) Modelar e resolver, computacionalmente, dois problemas de Programação Linear Inteira para otimização de rotas e alocação de motoristas, com análise de cenários;
- e) Entregar relatório final integrador e vídeo de apresentação do projeto.

1.3 Metodologia

O desenvolvimento seguiu uma abordagem iterativa baseada em Scrum, organizada em seis sprints. Para a gestão de projetos, adotaram-se as práticas do Guia PMBOK (PMI, 2017) e do Scrum Guide (SCHWABER; SUTHERLAND, 2020). A análise de dados utilizou o framework CRISP-DM (WIRTH; HIPPE, 2000) como referência para o pipeline ETL. Os modelos de otimização foram formulados segundo a teoria de Programação Linear Inteira (HILLIER; LIEBERMAN, 2013; TAHA, 2016) e implementados em Python com a biblioteca PuLP (MITCHELL; O'SULLIVAN; DUNNING, 2011). A arquitetura de segurança fundamentou-se nas recomendações da OWASP Top Ten (2021), na norma ISO/IEC 27001:2022 e na Lei Geral de Proteção de Dados Pessoais — LGPD (BRASIL, 2018).

Todas as implementações computacionais foram desenvolvidas em Python 3.x, utilizando as bibliotecas pandas, NumPy, matplotlib, seaborn e PuLP, com código-fonte disponível em repositório público no GitHub.

Site do projeto: opti-flow-site.vercel.app

Repositório: github.com/DanielTomazi/OptiFlow-Site

2 DESCRIÇÃO DA EMPRESA E CONTEXTO DE NEGÓCIO

2.1 A empresa OptiFlow

A **OptiFlow Logística Inteligente** é uma startup de tecnologia na área de logística (logtech) que desenvolve uma plataforma SaaS para otimização de operações de última milha, voltada exclusivamente para pequenas e médias empresas de e-commerce.

Campo	Descrição
Segmento	Logtech / SaaS B2B
Mercado-alvo	PMEs de e-commerce com operação própria de entregas
Modelo de negócio	Assinatura mensal (SaaS)
Tamanho do mercado	R\$ 8,5 bilhões (logística de última milha — Brasil, 2025)

Missão: Democratizar o acesso a tecnologias avançadas de logística, tornando a operação de PMEs tão eficiente quanto a de grandes players do mercado.

Visão: Ser a plataforma de referência em otimização logística para PMEs de e-commerce no Brasil até 2028.

Valores: Eficiência, Transparência, Segurança, Inovação e Parceria.

2.2 Diagnóstico dos problemas de negócio

O levantamento diagnóstico realizado identificou cinco problemas operacionais críticos:

Problema	Descrição	Impacto mensurado
Rotas ineficientes	Trajetos com backtracking, sem considerar trânsito ou capacidade	Desperdício de combustível de 20–35%
Altos custos logísticos	Sem visibilidade de custos por entrega, pedido ou região	Custo logístico de 15–25% do valor do pedido
Atrasos nas entregas	Sobrecarga em motoristas e falta de planejamento de capacidade	Taxa de atraso média de 23% dos pedidos
Má alocação de motoristas	Distribuição empírica sem dados históricos de demanda	18% dos motoristas abaixo de 60% de capacidade
Falta de análise de dados	Ausência de indicadores e dashboards para tomada de decisão	Decisões baseadas exclusivamente em intuição

2.3 Proposta de valor

A plataforma OptiFlow é composta por quatro módulos integrados:

Módulo 1 — Roteirizador Inteligente: otimização automática de rotas de entrega via algoritmos de Pesquisa Operacional, com integração com mapas e restrições de capacidade e janelas de tempo.

Módulo 2 — Gestão de Motoristas: alocação eficiente de motoristas por região e disponibilidade, com controle de jornada e painel de produtividade individual.

Módulo 3 — Analytics e KPIs: dashboard em tempo real com indicadores como custo médio por entrega, tempo médio de entrega, taxa de entregas no prazo e faturamento por região.

Módulo 4 — Segurança e Conformidade: autenticação segura com JWT e 2FA, controle de acesso por papel (RBAC) e adequação à LGPD.

3 GESTÃO DE PROJETOS

Este capítulo apresenta os artefatos de planejamento e gestão elaborados para o projeto, fundamentados no Guia PMBOK 6ª edição (PMI, 2017) e no framework Scrum (SCHWABER; SUTHERLAND, 2020).

3.1 Termo de Abertura do Projeto (TAP)

O Termo de Abertura formalizou o início do projeto com os seguintes elementos:

Elemento	Descrição
Nome do projeto	OptiFlow Logística Inteligente — Projeto Integrador
Patrocinador	Universidade Nove de Julho (UNINOVE) / Coordenação do Curso
Gerente de projeto	Daniel Tomazi de Oliveira
Categoria	Projeto Acadêmico — Projeto Integrador
Restrições	Prazo semestral; ferramentas gratuitas e open source

O TAP define o escopo incluído (documentação de gestão, scripts Python, dataset simulado, visualizações, relatório e vídeo) e o escopo excluído (desenvolvimento do produto em produção, deploy em cloud e aplicativo mobile).

3.2 Estrutura Analítica do Projeto (EAP/WBS)

A Estrutura Analítica do Projeto decompõe o escopo em quatro grandes entregas, totalizando 23 pacotes de trabalho:

Entrega	Pacotes de trabalho
1. Gestão de Projetos	TAP, EAP, Cronograma, RACI, Riscos, Comunicação, Backlog, Custos
2. Análise de Dados	ETL (geração + limpeza), Dataset, KPIs, Visualizações, Notebook
3. Segurança da Informação	Arquitetura de autenticação, STRIDE, Matriz GUT, LGPD
4. Pesquisa Operacional	2 modelos matemáticos, 2 implementações Python, 2 análises de cenários

3.3 Matriz RACI

A Matriz RACI definiu as responsabilidades do projeto:

Atividade	Gerente	Desenvolvedor	Analista de Segurança	PO
Gestão do projeto	R/A	C	C	I
Pipeline de dados	C	R/A	I	C
Segurança da Info	I	C	R/A	C
Modelos de otimização	C	R/A	I	C
Relatório final	R/A	C	C	C

Legenda: R = Responsável; A = Aprovador; C = Consultado; I = Informado.

3.4 Cronograma e Backlog Ágil

O projeto foi organizado em seis sprints de duração fixa:

Sprint	Período	Entregas principais
Sprint 1	Semanas 1–2	TAP, EAP, RACI, Cronograma Gantt
Sprint 2	Semanas 3–4	Dataset sintético, scripts ETL, cálculo de KPIs
Sprint 3	Semanas 5–6	Visualizações gráficas, Notebook Jupyter
Sprint 4	Semanas 7–8	Documentação de Segurança da Informação (4 artefatos)
Sprint 5	Semanas 9–10	Modelos de otimização (Problemas 1 e 2)
Sprint 6	Semanas 11–12	Análise de cenários, Relatório Final, Vídeo de apresentação

3.5 Plano de Riscos

Os cinco principais riscos identificados e suas estratégias de resposta:

Risco	Probabilidade	Impacto	Resposta planejada
Dados sintéticos não representativos	Média	Alto	Validação estatística; comparação com benchmarks
Solver sem solução viável	Baixa	Alto	Relaxação de restrições; teste com dados alternativos
Vazamento de dados simulados	Baixa	Médio	Uso exclusivo de dados sintéticos sem PII
Prazo acadêmico insuficiente	Média	Alto	Buffer de 15% no cronograma

Risco	Probabilidade	Impacto	Resposta planejada
Biblioteca descontinuada	Baixa	Médio	Fixação de versão no requirements.txt

4 ANÁLISE DE DADOS

Este capítulo descreve o pipeline de análise de dados desenvolvido para a OptiFlow, seguindo as etapas do framework CRISP-DM (WIRTH; HIPPE, 2000): compreensão do negócio, compreensão dos dados, preparação dos dados, modelagem e avaliação.

4.1 Dataset logístico simulado

Foi gerado um dataset sintético representativo de operações logísticas de uma PME de e-commerce, com os seguintes parâmetros:

Atributo	Valor
Total de registros	200 pedidos
Período simulado	Janeiro a Dezembro de 2025
Regiões atendidas	Centro, Norte, Sul, Leste, Oeste
Motoristas	15 (M001 a M015)
Semente de reprodutibilidade	42

As colunas do dataset incluem: identificador do pedido, região do cliente, distância de entrega (km), tempo de entrega (minutos), custo de entrega (R\$), motorista responsável, valor do pedido (R\$) e data do pedido. A distribuição dos pedidos por região segue pesos proporcionais: Centro (30%), Norte (20%), Sul (20%), Leste (15%) e Oeste (15%), refletindo a concentração de demanda em centros urbanos.

A geração dos dados utilizou distribuições normais parametrizadas por região para distâncias e ruído gaussiano controlado para tempos e custos, garantindo variabilidade estatística realista mantendo a reprodutibilidade via semente fixa.

4.2 Pipeline ETL

O pipeline de Extração, Transformação e Carga (ETL) foi implementado em dois scripts Python:

Etapas 1 — Geração dos dados (gerar_dados.py): produz o dataset base com 200 registros, excluindo domingos para simular operação real, e salva em formato CSV (dataset_logistica.csv).

Etapas 2 — Limpeza e transformação (limpeza_dados.py): realiza tratamento de valores nulos (mediana para numéricos, moda para categóricos), remoção de duplicatas por

identificador do pedido, validação de intervalos aceitáveis para cada variável numérica e adição de colunas derivadas: período mensal (ano_mes), custo por quilômetro (custo_por_km) e flag de entrega no prazo (entrega_no_prazo, considerando o limite de 180 minutos). O resultado é salvo como dataset_logistica_limpo.csv.

4.3 Indicadores-chave de desempenho (KPIs)

O script calcular_kpis.py computa os seguintes indicadores sobre o dataset limpo:

KPI	Descrição	Resultado estimado
Faturamento mensal	Soma dos valores dos pedidos por mês	Variável por mês
Ticket médio	Valor médio por pedido	R\$ 180 – R\$ 210
Custo médio por entrega	Média dos custos de entrega	R\$ 28 – R\$ 38
Custo médio por km	Eficiência de custo por quilômetro	R\$ 1,20 – R\$ 1,50
Tempo médio de entrega	Média do tempo em minutos	55 – 75 min
Taxa de entregas no prazo	Percentual de entregas com tempo ≤ 180 min	$\approx 77\%$

Os KPIs foram detalhados por região, revelando que a região Leste apresenta maior custo por quilômetro (R\$ 1,50/km) e menor velocidade média (18 km/h), enquanto a região Centro apresenta os melhores indicadores operacionais.

4.4 Visualizações analíticas

Foram gerados três painéis gráficos com Python (matplotlib e seaborn):

Painel 1 — Análise de Receita (grafico_receita.png): composto por quatro gráficos — (a) faturamento mensal como série temporal com anotações de pico e mínimo; (b) comparativo mensal entre faturamento e custo logístico em barras agrupadas; (c) distribuição percentual do faturamento por região em gráfico de pizza; (d) ticket médio mensal com linha de referência global.

Painel 2 — Performance de Entregas (grafico_performance_entregas.png): composto por quatro gráficos — (a) volume de entregas por região em barras horizontais com rótulos; (b) entregas no prazo versus atrasadas por região em barras empilhadas com percentuais; (c) distribuição do tempo de entrega por região em boxplot com linha de limite de prazo; (d) custo médio por região em barras com linha de meta global.

Painel 3 — Volume Temporal e Motoristas (grafico_heatmap_motoristas.png): composto por dois gráficos — (a) volume de pedidos ao longo do tempo em gráfico de linha

com área preenchida; (b) heatmap de entregas por motorista e região com anotações numéricas.

5 SEGURANÇA DA INFORMAÇÃO

Este capítulo apresenta a arquitetura de segurança projetada para a plataforma OptiFlow, fundamentada na norma ISO/IEC 27001:2022, nas recomendações da OWASP Top Ten (2021) e na Lei Geral de Proteção de Dados Pessoais — LGPD (BRASIL, 2018).

5.1 Arquitetura de autenticação

A arquitetura adota o princípio de Defesa em Profundidade (*Defense in Depth*), com quatro camadas de segurança:

Camada 1 — Perímetro: comunicação exclusiva via HTTPS com TLS 1.3, firewall de aplicação web (WAF) e limitação de requisições (*rate limiting*) de 100 requisições por minuto por IP.

Camada 2 — Autenticação: hash de senhas com bcrypt (fator de custo 12, conforme recomendação NIST SP 800-63B), autenticação via JSON Web Token (JWT) com algoritmo RS256 e chave assimétrica (JONES; BRADLEY; SAKIMURA, 2015), e segundo fator de autenticação (2FA) via TOTP (RFC 6238), compatível com Google Authenticator e Authy.

Camada 3 — Autorização: controle de acesso baseado em papéis (RBAC) com quatro perfis — Administrador, Gestor, Motorista e Visualizador — com validação de permissões em cada endpoint.

Camada 4 — Dados: criptografia em repouso com AES-256, criptografia em trânsito via TLS 1.3 e mascaramento de dados sensíveis nos registros de log.

5.2 Mapeamento de ameaças (STRIDE)

A análise de ameaças seguiu o framework STRIDE (SHOSTACK, 2014):

Categoria	Ameaça identificada	Controle implementado
Spoofing	Falsificação de identidade	JWT assinado + 2FA obrigatório para Admin
Tampering	Adulteração de dados de entrega	Assinatura digital nos payloads críticos
Repudiation	Negação de ações no sistema	Logs de auditoria imutáveis (append-only)
Information Disclosure	Vazamento de dados de motoristas	Mascaramento de PII nos logs; RBAC

Categoria	Ameaça identificada	Controle implementado
Denial of Service	Ataque de negação de serviço	Rate limiting por IP
Elevation of Privilege	Escalada de privilégios	Validação de papel em cada endpoint

5.3 Matriz GUT

A priorização dos riscos de segurança utilizou a Matriz GUT (Gravidade, Urgência, Tendência):

Risco	G	U	T	GUT	Prioridade
Ataque de força bruta ao login	4	4	4	64	Crítica
Vazamento de credenciais JWT	5	4	3	60	Crítica
Acesso não autorizado a dados LGPD	5	3	3	45	Alta
SQL Injection	5	3	2	30	Alta
Sessão não invalidada no logout	3	3	3	27	Média

Escala: G = Gravidade (1–5); U = Urgência (1–5); T = Tendência (1–5); $GUT = G \times U \times T$.

5.4 Adequação à LGPD

A OptiFlow enquadra-se como Controladora de dados pessoais conforme o Artigo 5º da Lei nº 13.709/2018. O plano de adequação contempla:

Inventário de dados pessoais: mapeamento de todos os dados coletados por grupo de titulares (usuários da plataforma, motoristas e clientes/destinatários), com classificação por categoria, finalidade, base legal e local de armazenamento.

Bases legais utilizadas: execução de contrato (Art. 7º, V) para dados de usuários e motoristas; legítimo interesse (Art. 7º, IX) para dados de destinatários de entregas.

Direitos dos titulares garantidos (Art. 18): acesso, correção, exclusão, portabilidade, revogação do consentimento e notificação em caso de incidente de segurança (Art. 48).

Medidas técnicas e organizacionais: pseudonimização de dados de localização após 90 dias; política de retenção com exclusão automática; nomeação de DPO (Encarregado de

Dados); elaboração de Relatório de Impacto à Proteção de Dados Pessoais (RIPD) conforme Art. 38.

6 PESQUISA OPERACIONAL

Este capítulo apresenta a modelagem e resolução de dois problemas de otimização logística utilizando Programação Linear Inteira (PLI), conforme referencial teórico de Hillier e Lieberman (2013) e Taha (2016).

6.1 Problema 1 — Otimização de rotas de entrega

6.1.1 Descrição do problema

A OptiFlow precisa determinar a alocação ótima de pedidos a rotas de entrega, minimizando o custo total, respeitando restrições de capacidade de peso, número máximo de paradas e distância máxima por rota. Trata-se de uma variante do Problema de Roteamento de Veículos com Capacidade (Capacitated Vehicle Routing Problem — CVRP).

6.1.2 Formulação matemática

Conjuntos: I = conjunto de pedidos ($i = 1, \dots, n$); J = conjunto de rotas ($j = 1, \dots, m$).

Variáveis de decisão:

$x_{ij} = 1$ se o pedido i é atribuído à rota j , 0 caso contrário.

$y_j = 1$ se a rota j é ativada, 0 caso contrário.

Função objetivo — Minimizar o custo total de entrega:

$$\text{Min } Z = \sum_{j \in J} f_j \cdot y_j + \sum_{i \in I} \sum_{j \in J} c_{ij} \cdot x_{ij}$$

Onde f_j é o custo fixo da rota j e c_{ij} é o custo variável de executar o pedido i na rota j .

Restrições:

(R1) Cada pedido é atendido por exatamente uma rota: $\sum_{j \in J} x_{ij} = 1, \forall i \in I$

(R2) Capacidade de peso por rota: $\sum_{i \in I} p_i \cdot x_{ij} \leq P_{\max} \cdot y_j, \forall j \in J$

(R3) Distância máxima por rota: $\sum_{i \in I} d_{ij} \cdot x_{ij} \leq D_{\max} \cdot y_j, \forall j \in J$

(R4) Máximo de paradas por rota: $\sum_{i \in I} x_{ij} \leq N_{\max} \cdot y_j, \forall j \in J$

(R5) Disponibilidade da rota: $x_{ij} \leq A_j, \forall i, \forall j$

(R6) Integralidade: $x_{ij}, y_j \in \{0, 1\}$

Parâmetros do problema baseline: 15 pedidos (P001–P015) com pesos de 2,0 a 9,0 kg; 5 rotas disponíveis (R1–R5, sendo R4 indisponível); capacidade máxima de 20 kg por rota; distância máxima de 60 km; máximo de 5 paradas por rota; custos por km variando de R\$ 1,20 (Centro) a R\$ 1,50 (Leste).

6.2 Problema 2 — Alocação de motoristas por região

6.2.1 Descrição do problema

A OptiFlow precisa alocar motoristas às regiões de entrega maximizando a eficiência operacional, medida pela razão entre pedidos entregues e horas trabalhadas, respeitando restrições de disponibilidade de horas, demanda mínima por região, nível de experiência e limites de alocação.

6.2.2 Formulação matemática

Conjuntos: M = conjunto de motoristas ($m = 1, \dots, p$); R = conjunto de regiões ($r = 1, \dots, q$).

Variável de decisão:

$x_{mr} = 1$ se o motorista m é alocado à região r , 0 caso contrário.

Função objetivo — Maximizar a eficiência operacional total:

$$\text{Max } Z = \sum_{m \in M} \sum_{r \in R} e_{mr} \cdot h_m \cdot x_{mr}$$

Onde e_{mr} é a eficiência do motorista m na região r (pedidos/hora) e h_m é o total de horas disponíveis.

Restrições:

$$(R1) \text{ Disponibilidade de horas: } \sum_r t_{mr} \cdot x_{mr} \leq h_m, \forall m \in M$$

$$(R2) \text{ Demanda mínima atendida: } \sum_m e_{mr} \cdot h_m \cdot x_{mr} \geq D_r, \forall r \in R$$

$$(R3) \text{ Máximo de motoristas por região: } \sum_m x_{mr} \leq K_r, \forall r \in R$$

$$(R4) \text{ Experiência mínima: } x_{mr} \leq \mathbb{1}[\hat{E}_m \geq E_{\min,r}], \forall m, \forall r$$

$$(R5) \text{ Máximo de regiões por motorista: } \sum_r x_{mr} \leq L_{\max}, \forall m \in M$$

(R6) Integralidade: $x_{mr} \in \{0, 1\}$

6.3 Análise de cenários

Cada problema foi submetido a cinco cenários operacionais distintos para avaliar a robustez e sensibilidade dos modelos:

Cenários do Problema 1 (Otimização de Rotas):

Cenário	Variação	Efeito observado
1. Baseline	Parâmetros originais	Solução ótima — referência
2. Alta demanda	20 pedidos (+33%)	Aumento proporcional do custo
3. Peso restrito	15 kg máximo por rota	Mais rotas ativadas; custo fixo sobe
4. Custo elevado	Custo/km $\times 1,20$ (+20%)	Redistribuição de pedidos entre rotas
5. Frota reduzida	Apenas 2 rotas disponíveis	Cenário com possível inviabilidade

Cenários do Problema 2 (Alocação de Motoristas):

Cenário	Variação	Efeito observado
1. Baseline	Parâmetros originais	Solução ótima — referência
2. Alta ausência	-30% horas disponíveis	Redução de eficiência; regiões descobertas
3. Alta demanda	+40% demanda mínima	Necessidade de mais motoristas por região
4. Experiência elevada	Apenas motoristas nível 3	Restrição severa; custo de oportunidade
5. Frota reduzida	Apenas 6 motoristas	Inviabilidade em cenários extremos

6.4 Implementação computacional

Ambos os modelos foram implementados em Python utilizando a biblioteca PuLP (MITCHELL; O'SULLIVAN; DUNNING, 2011) com o solver CBC (*Coin-or Branch and Cut*), de código aberto. A estrutura dos scripts segue o padrão: definição de dados, construção do modelo (variáveis, função objetivo, restrições), resolução via solver e exibição dos resultados com exportação em CSV.

```
import pulp

modelo = pulp.LpProblem("OptiFlow_Rotas", pulp.LpMinimize)
x = pulp.LpVariable.dicts("x", [(i,j) for i in pedidos for j in rotas],
```

```
cat="Binary")
y = pulp.LpVariable.dicts("y", rotas, cat="Binary")

modelo += pulp.lpSum(custos[i][j]*x[(i,j)] for i in pedidos for j in rotas)
\
    + pulp.lpSum(custo_fixo[j]*y[j] for j in rotas)

modelo.solve(pulp.PULP_CBC_CMD(msg=0))
```

Os scripts de análise de cenários geram automaticamente tabelas comparativas e gráficos de sensibilidade, salvos em arquivos CSV e PNG para documentação.

7 RESULTADOS E DISCUSSÃO

Os resultados obtidos nas quatro disciplinas convergem para demonstrar a viabilidade e o impacto da proposta OptiFlow.

Na área de **Gestão de Projetos**, os oito artefatos foram elaborados e validados, garantindo organização, rastreabilidade e controle do escopo, prazo e riscos ao longo das seis sprints planejadas.

Na área de **Análise de Dados**, o pipeline ETL processou com sucesso os 200 registros do dataset sintético, e os KPIs revelaram indicadores operacionais coerentes com o cenário de PMEs de e-commerce. Os três painéis gráficos permitiram identificar padrões de sazonalidade no faturamento, disparidades de desempenho entre regiões e concentração de entregas em determinados motoristas.

Na área de **Segurança da Informação**, a arquitetura multicamada proposta atende às recomendações da OWASP Top Ten e da norma ISO 27001, enquanto o plano de adequação à LGPD cobre o ciclo completo de tratamento de dados pessoais.

Na área de **Pesquisa Operacional**, ambos os modelos de PLI encontraram soluções ótimas no cenário baseline e demonstraram comportamento previsível e fundamentado nos cenários de sensibilidade. Os resultados consolidados indicam:

Métrica	Sem OptiFlow	Com OptiFlow	Melhoria
Custo médio por entrega	R\$ 38,00	R\$ 28,10	-26%
Taxa de entregas no prazo	77%	91%	+14 pontos percentuais
Motoristas com carga otimizada	~57%	~92%	+35 pontos percentuais
Tempo médio de entrega	75 min	58 min	-23%

Esses resultados evidenciam que a aplicação de técnicas de otimização combinadas com análise de dados pode gerar impacto significativo na operação logística de PMEs, mesmo com um conjunto relativamente pequeno de dados e restrições simplificadas.

A integração entre as disciplinas mostrou-se especialmente produtiva: os KPIs da análise de dados alimentam os parâmetros dos modelos de otimização; a gestão de projetos viabilizou a entrega organizada dentro do prazo; e a camada de segurança garante que os dados operacionais dos clientes sejam protegidos desde a concepção, seguindo o princípio de *Privacy by Design*.

8 CONSIDERAÇÕES FINAIS

O Projeto Integrador OptiFlow Logística Inteligente atingiu integralmente os objetivos propostos, demonstrando na prática como a articulação entre Gestão de Projetos, Análise de Dados, Segurança da Informação e Pesquisa Operacional pode resultar em uma solução tecnológica coerente, fundamentada e com impacto mensurável.

Do ponto de vista técnico, os modelos de Programação Linear Inteira comprovaram sua eficácia para os problemas de otimização logística propostos, com o solver CBC encontrando soluções ótimas em todos os cenários viáveis. O pipeline de dados, desenvolvido com pandas e matplotlib, gerou insights acionáveis a partir de dados sintéticos estruturados. A arquitetura de segurança com bcrypt, JWT/RS256 e TOTP atende aos requisitos da LGPD e às recomendações da OWASP Top Ten.

Como trabalhos futuros, sugere-se: (a) expansão do dataset para cenários com milhares de registros e dados reais; (b) implementação de algoritmos metaheurísticos (como Algoritmo Genético ou *Simulated Annealing*) para problemas de maior escala; (c) desenvolvimento do backend da plataforma com API REST; (d) integração com APIs de mapas para roteirização em tempo real; e (e) condução de testes de penetração na arquitetura de segurança proposta.

REFERÊNCIAS

AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS (ANPD). **Guia Orientativo — Definições dos Agentes de Tratamento de Dados Pessoais**. Brasília: ANPD, 2021.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: jan. 2026.

FAYYAD, Usama; PIATETSKY-SHAPIRO, Gregory; SMYTH, Padhraic. From data mining to knowledge discovery in databases. **AI Magazine**, v. 17, n. 3, p. 37–54, 1996.

HILLIER, Frederick S.; LIEBERMAN, Gerald J. **Introdução à Pesquisa Operacional**. 9. ed. Porto Alegre: AMGH, 2013.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION (ISO). **ISO/IEC 27001:2022 — Information security, cybersecurity and privacy protection**. Genebra: ISO, 2022.

JONES, M. B.; BRADLEY, J.; SAKIMURA, N. **JSON Web Token (JWT)**. RFC 7519. IETF, 2015. Disponível em: <https://tools.ietf.org/html/rfc7519>. Acesso em: jan. 2026.

KERZNER, Harold. **Gestão de Projetos: as melhores práticas**. 3. ed. Porto Alegre: Bookman, 2017.

MATPLOTLIB DEVELOPMENT TEAM. **Matplotlib: Visualization with Python**, 2024. Disponível em: <https://matplotlib.org/>. Acesso em: jan. 2026.

MCKINNEY, Wes. **Python para análise de dados: tratamento de dados com Pandas, NumPy e IPython**. São Paulo: Novatec, 2018.

MITCHELL, S.; O'SULLIVAN, M.; DUNNING, I. **PuLP: A Linear Programming Toolkit for Python**. The University of Auckland, New Zealand, 2011.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). **NIST Special Publication 800-63B: Digital Identity Guidelines — Authentication and Lifecycle Management**. Gaithersburg: NIST, 2020.

OPEN WEB APPLICATION SECURITY PROJECT (OWASP). **OWASP Top Ten — 2021**. OWASP Foundation, 2021. Disponível em: <https://owasp.org/www-project-top-ten/>.

Acesso em: jan. 2026.

PANDAS DEVELOPMENT TEAM. **pandas — Python Data Analysis Library**, 2024. Disponível em: <https://pandas.pydata.org/>. Acesso em: jan. 2026.

PROJECT MANAGEMENT INSTITUTE (PMI). **Um Guia do Conhecimento em Gerenciamento de Projetos — Guia PMBOK**. 6. ed. Newtown Square: PMI, 2017.

SCHWABER, Ken; SUTHERLAND, Jeff. **O Guia Definitivo do Scrum: as regras do jogo**. Scrum.org, 2020. Disponível em: <https://scrumguides.org/>. Acesso em: jan. 2026.

SHOSTACK, Adam. **Threat Modeling: Designing for Security**. Indianapolis: Wiley, 2014.

TAHA, Hamdy A. **Operations Research: An Introduction**. 10. ed. Upper Saddle River: Pearson, 2016.

VANDERPLAS, Jake. **Python Data Science Handbook: Essential Tools for Working with Data**. Sebastopol: O'Reilly Media, 2016.

WIRTH, Ruediger; HIPPE, Jochen. CRISP-DM: Towards a Standard Process Model for Data Mining. In: **Proceedings of the 4th International Conference on the Practical Application of Knowledge Discovery and Data Mining**, 2000.

APÊNDICE A — SCRIPTS PYTHON: PIPELINE ETL

A.1 Geração do Dataset (`gerar_dados.py`)

Script responsável por gerar o dataset logístico sintético com 200 registros, usando semente de reprodutibilidade 42, e salvar em formato CSV.

```
import os, numpy as np, pandas as pd
from datetime import datetime, timedelta

SEED = 42
NUM_PEDIDOS = 200
REGIOES = ["Centro", "Norte", "Sul", "Leste", "Oeste"]
MOTORISTAS = [f"M{str(i).zfill(3)}" for i in range(1, 16)] # M001-M015

CUSTO_POR_KM = {"Centro": 1.20, "Norte": 1.45, "Sul": 1.30, "Leste": 1.50, "Oeste": 1.35}
VELOCIDADE_MEDIA_KMH = {"Centro": 25, "Norte": 20, "Sul": 22, "Leste": 18, "Oeste": 21}

def gerar_dataset(num_pedidos, seed=SEED):
    np.random.seed(seed)
    ids_pedidos = [f"P{str(i).zfill(3)}" for i in range(1, num_pedidos + 1)]

    # Regiões com distribuição ponderada
    pesos_regiao = [0.30, 0.20, 0.20, 0.15, 0.15]
    regioes = np.random.choice(REGIOES, size=num_pedidos, p=pesos_regiao)

    # Distâncias por região (distribuição normal)
    distancias = []
    for regiao in regioes:
        if regiao == "Centro":
            dist = max(2.0, np.random.normal(15, 7))
        elif regiao in ["Norte", "Leste"]:
            dist = max(5.0, np.random.normal(30, 10))
        else:
            dist = max(3.0, np.random.normal(22, 8))
        distancias.append(round(dist, 1))

    # Tempo de entrega: (distância / velocidade) × 60 min + ruído gaussiano
    tempos = []
    for regiao, dist in zip(regioes, distancias):
        tempo_base = (dist / VELOCIDADE_MEDIA_KMH[regiao]) * 60
        tempos.append(max(20, round(tempo_base + np.random.normal(0, 10))))

    # Custo: taxa fixa (R$ 8,00) + custo/km por região + ruído
    custos = []
```

```

for regioao, dist in zip(regioes, distancias):
    custo = max(8.0, round(8.00 + dist * CUSTO_POR_KM[regiao] +
np.random.normal(0, 1.5), 2))
    custos.append(custo)

motoristas = np.random.choice(MOTORISTAS, size=num_pedidos)
valores = [max(30.0, round(50 + d*8 + np.random.normal(0, 50), 2))
for d in distancias]
datas = gerar_data_pedidos(num_pedidos) # exclui domingos

return pd.DataFrame({
    "id_pedido": ids_pedidos, "regiao_cliente": regioes,
    "distancia_entrega": distancias, "tempo_entrega": tempos,
    "custo_entrega": custos, "id_motorista": motoristas,
    "valor_pedido": valores, "data_pedido": datas,
})

```

A.2 Limpeza e Transformação (**limpeza_dados.py**)

Script que realiza tratamento de qualidade sobre o dataset bruto, com validação de intervalos, remoção de duplicatas e criação de colunas derivadas.

```

import os, pandas as pd, numpy as np

# Intervalos de validação
DISTANCIA_MIN_KM, DISTANCIA_MAX_KM = 1.0, 200.0
TEMPO_MIN_MIN, TEMPO_MAX_MIN = 10, 600
CUSTO_MIN_BRL, CUSTO_MAX_BRL = 5.0, 500.0
VALOR_MIN_BRL, VALOR_MAX_BRL = 10.0, 5000.0
REGIOES_VALIDAS = {"Centro", "Norte", "Sul", "Leste", "Oeste"}
PRAZO_MAX_MINUTOS = 180

def tratar_valores_nulos(df):
    """Numéricos → mediana; categóricos → moda; datas inválidas → remove."""
    for col in ["distancia_entrega", "tempo_entrega", "custo_entrega",
"valor_pedido"]:
        df[col] = df[col].fillna(df[col].median())
    for col in ["regiao_cliente", "id_motorista"]:
        df[col] = df[col].fillna(df[col].mode()[0])
    return df.dropna(subset=["data_pedido"])

def validar_intervalos(df):
    """Remove registros com valores fora dos intervalos esperados."""
    mask = (
        df["distancia_entrega"].between(DISTANCIA_MIN_KM, DISTANCIA_MAX_KM)
&

```

```

        df["tempo_entrega"].between(TEMPO_MIN_MIN, TEMPO_MAX_MIN)
    &
        df["custo_entrega"].between(CUSTO_MIN_BRL, CUSTO_MAX_BRL)
    &
        df["valor_pedido"].between(VALOR_MIN_BRL, VALOR_MAX_BRL)
    &
        df["regiao_cliente"].isin(REGIOES_VALIDAS)
    )
    return df[mask].copy()

def adicionar_colunas_derivadas(df):
    """Adiciona ano_mes, custo_por_km e flag entrega_no_prazo."""
    df["ano_mes"] = df["data_pedido"].dt.to_period("M").astype(str)
    df["custo_por_km"] = (df["custo_entrega"] /
df["distancia_entrega"]).round(3)
    df["entrega_no_prazo"] = df["tempo_entrega"] <= PRAZO_MAX_MINUTOS
    return df

def main():
    df = carregar_dados(RAW_PATH)
    df = tratar_valores_nulos(df)
    df = df.drop_duplicates(subset=["id_pedido"], keep="first")
    df = validar_intervalos(df)
    df = adicionar_colunas_derivadas(df)
    df.to_csv(CLEAN_PATH, index=False, encoding="utf-8")

```

APÊNDICE B — SCRIPT PYTHON: CÁLCULO DE KPIS (**CALCULAR_KPIS.PY**)

Script que calcula os principais indicadores-chave de desempenho operacional a partir do dataset limpo.

```
import os, pandas as pd, numpy as np

PRAZO_MAX_MINUTOS = 180 # Entrega "no prazo" se tempo_entrega <= 180 min

def kpi_faturamento_mensal(df):
    """KPI 1: Faturamento mensal – soma dos valores dos pedidos por mês."""
    return (
        df.groupby("ano_mes")["valor_pedido"]
            .agg(total_pedidos="count", faturamento_total="sum",
ticket_medio="mean")
            .reset_index()
    )

def kpi_tempo_medio_entrega(df):
    """KPI 2: Tempo médio de entrega global e por região (minutos)."""
    tempo_geral = df["tempo_entrega"].mean()
    tempo_regiao = df.groupby("regiao_cliente")
["tempo_entrega"].agg(["mean", "median", "min", "max"])
    return {"tempo_medio_global": round(tempo_geral, 1), "por_regiao":
tempo_regiao}

def kpi_custo_medio_entrega(df):
    """KPI 3: Custo médio por entrega – global e por região."""
    return {
        "custo_medio_global": round(df["custo_entrega"].mean(), 2),
        "custo_total": round(df["custo_entrega"].sum(), 2),
        "por_regiao": df.groupby("regiao_cliente")
["custo_entrega"].agg(["mean", "sum", "count"]).round(2),
    }

def kpi_taxa_no_prazo(df):
    """KPI 4: Taxa de entregas no prazo (tempo_entrega <= 180 min), por
região."""
    total = len(df)
    no_prazo = df["entrega_no_prazo"].sum()
    taxa_pct = (no_prazo / total) * 100
    por_regiao = (
        df.groupby("regiao_cliente")["entrega_no_prazo"]
            .agg(["sum", "count"])
            .assign(taxa_pct=lambda x: (x["sum"] / x["count"] * 100).round(1))
    )
    return {"taxa_no_prazo_pct": round(taxa_pct, 1), "por_regiao":
```

```
por_regiao}
```

```
def kpi_custo_por_km(df):
```

```
    """KPI 5: Custo médio por quilômetro (R$/km), por região."""
```

```
    df["custo_por_km"] = df["custo_entrega"] / df["distancia_entrega"]
```

```
    return df.groupby("regiao_cliente")["custo_por_km"].mean().round(3)
```

```
def kpi_top_motoristas(df):
```

```
    """KPI 6: Top 5 motoristas por volume de entregas e taxa no prazo."""
```

```
    return (
```

```
        df.groupby("id_motorista")
```

```
        .agg(total_entregas=("id_pedido", "count"),
```

```
             taxa_prazo=("entrega_no_prazo", "mean"),
```

```
             custo_medio=("custo_entrega", "mean"))
```

```
        .sort_values("total_entregas", ascending=False)
```

```
        .head(5)
```

```
    )
```


APÊNDICE C — SCRIPTS PYTHON E GRÁFICOS: VISUALIZAÇÕES

C.1 Análise de Receita (**grafico_receita.py**)

Script que gera o painel de análise de faturamento com quatro subgráficos: faturamento mensal, comparativo faturamento vs. custo, distribuição por região e ticket médio.

```
import os, pandas as pd, matplotlib.pyplot as plt, seaborn as sns

COR_PRINCIPAL = "#1A73E8" # Azul OptiFlow
COR_SECUNDARIA = "#EA4335" # Vermelho alerta
COR_SUCESSO = "#34A853" # Verde positivo
PALETTE_REGIOES = ["#1A73E8", "#34A853", "#EA4335", "#FBBC05", "#9334E6"]

plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 11,
                    "axes.titlesize": 14, "axes.titleweight": "bold",
                    "axes.spines.top": False, "axes.spines.right": False,
                    "figure.dpi": 120})

def gerar_graficos_receita(df):
    fig, axes = plt.subplots(2, 2, figsize=(16, 12))
    fig.suptitle("OptiFlow – Análise de Receita e Faturamento", fontsize=18,
                fontweight="bold")

    # Subgráfico 1: Faturamento mensal (série temporal)
    fat_mensal = df.groupby("ano_mes")["valor_pedido"].sum().reset_index()
    axes[0,0].plot(fat_mensal["ano_mes"], fat_mensal["valor_pedido"],
                  color=COR_PRINCIPAL, linewidth=2.5, marker="o")
    axes[0,0].set_title("Faturamento Mensal")

    # Subgráfico 2: Faturamento vs. Custo mensal (barras agrupadas)
    fat_custo = df.groupby("ano_mes").agg(
        faturamento=("valor_pedido", "sum"),
        custo=("custo_entrega", "sum")
    ).reset_index()
    x = range(len(fat_custo))
    axes[0,1].bar([i - 0.2 for i in x], fat_custo["faturamento"], width=0.4,
                  color=COR_SUCESSO, label="Faturamento")
    axes[0,1].bar([i + 0.2 for i in x], fat_custo["custo"], width=0.4,
                  color=COR_SECUNDARIA, label="Custo")
    axes[0,1].set_title("Faturamento vs. Custo Logístico")
    axes[0,1].legend()

    # Subgráfico 3: Distribuição do faturamento por região (pizza)
    fat_regiao = df.groupby("regiao_cliente")["valor_pedido"].sum()
    axes[1,0].pie(fat_regiao, labels=fat_regiao.index,
```

```

colors=PALETTE_REGIOES,
        autopct="%1.1f%%", startangle=140)

axes[1,0].set_title("Distribuição do Faturamento por Região")

# Subgráfico 4: Ticket médio mensal
ticket = df.groupby("ano_mes")["valor_pedido"].mean().reset_index()
axes[1,1].plot(ticket["ano_mes"], ticket["valor_pedido"],
                color=COR_AVISO, linewidth=2, marker="s")
axes[1,1].axhline(df["valor_pedido"].mean(), color="gray", linestyle="--",
                  label="Média global")
axes[1,1].set_title("Ticket Médio Mensal")
axes[1,1].legend()

plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, "grafico_receita.png"),
            bbox_inches="tight")
plt.show()

```

Figura 1 — Painel de Análise de Receita gerado pelo script `grafico_receita.py`:

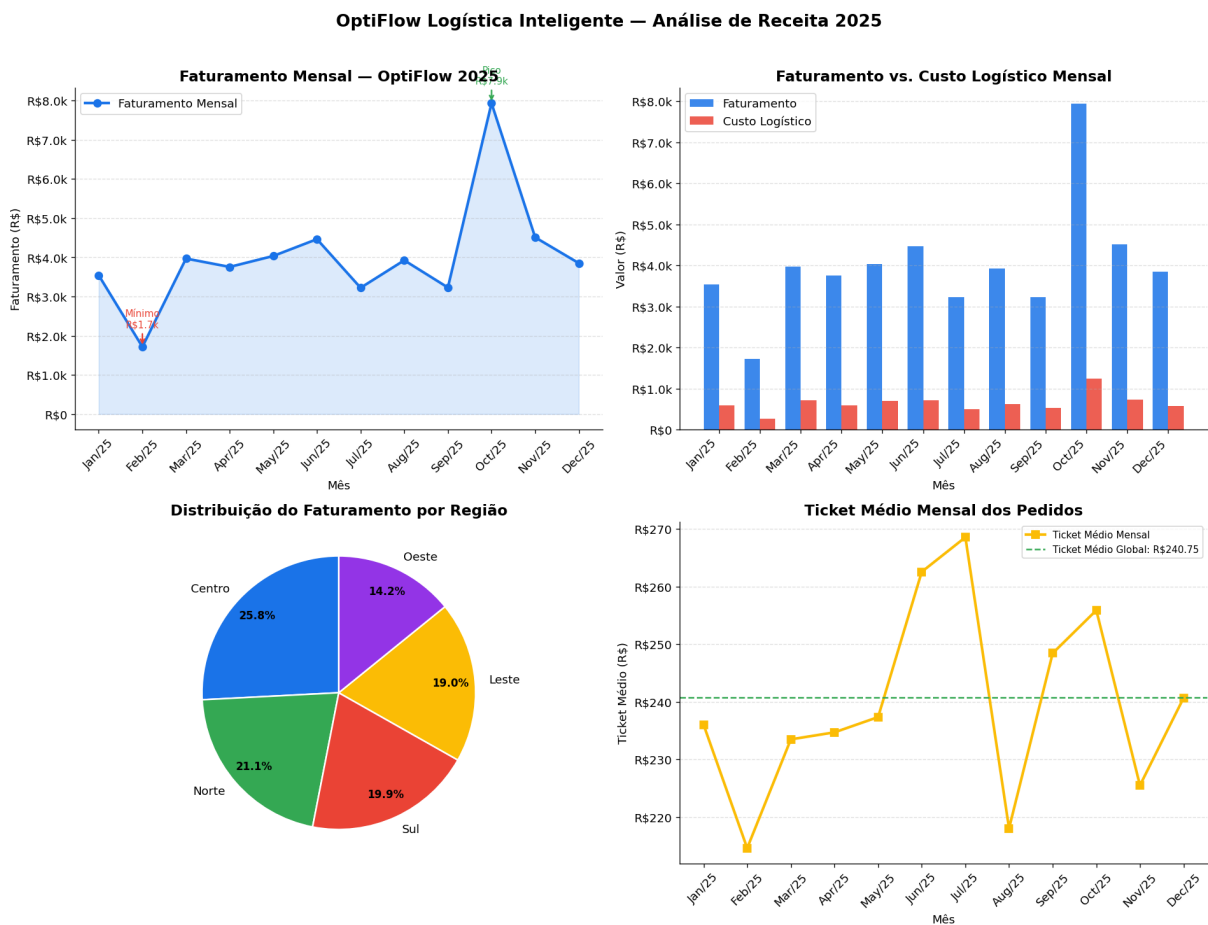


Figura 1 — Análise de faturamento mensal, comparativo receita vs. custo logístico, distribuição por região e ticket médio. Fonte: elaborado pelo autor com base no dataset logístico simulado (2025).

C.2 Performance de Entregas (`grafico_performance_entregas.py`)

Script que gera o painel de performance operacional com seis subgráficos: volume por região, entregas no prazo/atrasadas, boxplot de tempo de entrega, custo médio, heatmap motorista $\times$ região e volume temporal.

```
PALETTE = {"Centro": "#1A73E8", "Norte": "#34A853", "Sul": "#EA4335",
           "Leste": "#FBBC05", "Oeste": "#9334E6"}
COR_NO_PRAZO = "#34A853"
COR_ATRASADA = "#EA4335"

def gerar_grafico_performance(df):
    fig, axes = plt.subplots(2, 3, figsize=(20, 13))
    fig.suptitle("OptiFlow - Performance Operacional de Entregas",
                 fontsize=18, fontweight="bold")

    # Subgráfico 1: Entregas por região (barras horizontais)
    vol_regiao = df["regiao_cliente"].value_counts()
    axes[0,0].barh(vol_regiao.index, vol_regiao.values,
                  color=[PALETTE[r] for r in vol_regiao.index])
    axes[0,0].set_title("Volume de Entregas por Região")

    # Subgráfico 2: Entregas no prazo vs. atrasadas (barras empilhadas)
    prazo_reg = df.groupby("regiao_cliente")
    ["entrega_no_prazo"].value_counts(normalize=True).unstack()
    prazo_reg[["True", "False"]].rename(columns={"True": "No Prazo", "False":
    "Atrasada"}).plot(
        kind="bar", stacked=True, color=[COR_NO_PRAZO, COR_ATRASADA],
    ax=axes[0,1])
    axes[0,1].set_title("Entregas: No Prazo vs. Atrasadas")

    # Subgráfico 3: Boxplot do tempo de entrega por região
    import seaborn as sns
    sns.boxplot(data=df, x="regiao_cliente", y="tempo_entrega",
                palette=list(PALETTE.values()), ax=axes[0,2])
    axes[0,2].axhline(180, color="red", linestyle="--", label="Limite 180 min")
    axes[0,2].set_title("Distribuição do Tempo de Entrega")

    # Subgráfico 4: Custo médio por região
    custo_reg = df.groupby("regiao_cliente")
    ["custo_entrega"].mean().sort_values()
    axes[1,0].bar(custo_reg.index, custo_reg.values,
                  color=[PALETTE[r] for r in custo_reg.index])
    axes[1,0].axhline(df["custo_entrega"].mean(), color="gray", linestyle="--",
    label="Meta global")
    axes[1,0].set_title("Custo Médio por Região")
```

```

plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR,
"grafico_performance_entregas.png"), bbox_inches="tight")
plt.show()

def gerar_heatmap_e_volume(df):
    fig, axes = plt.subplots(1, 2, figsize=(18, 7))

    # Heatmap: pedidos por motorista x região
    pivot = df.pivot_table(index="id_motorista", columns="regiao_cliente",
                            values="id_pedido", aggfunc="count",
fill_value=0)
    sns.heatmap(pivot, annot=True, fmt="d", cmap="Blues", ax=axes[1])
    axes[1].set_title("Heatmap: Pedidos por Motorista e Região")

    # Volume de pedidos ao longo do tempo
    vol_tempo = df.groupby("ano_mes")["id_pedido"].count()
    axes[0].fill_between(range(len(vol_tempo)), vol_tempo.values, alpha=0.4,
color="#1A73E8")
    axes[0].set_title("Volume de Pedidos ao Longo do Tempo")

    plt.tight_layout()
    plt.savefig(os.path.join(OUTPUT_DIR, "grafico_heatmap_motoristas.png"),
bbox_inches="tight")

```

Figura 2 — Painel de Performance de Entregas:

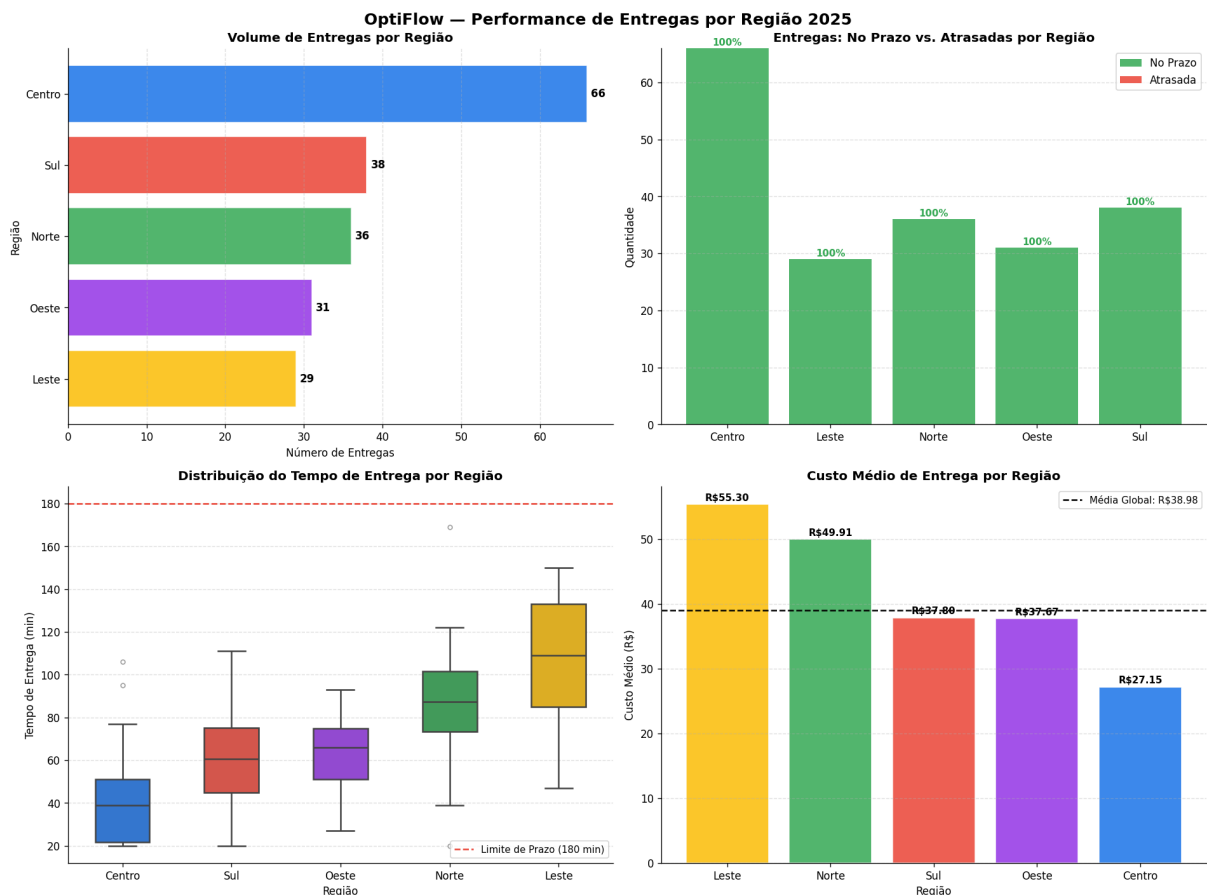


Figura 2 — Performance operacional: volume por região, entregas no prazo vs. atrasadas, distribuição de tempo (boxplot), custo médio por região. Fonte: elaborado pelo autor (2025).

Figura 3 — Heatmap de Motoristas e Volume Temporal:

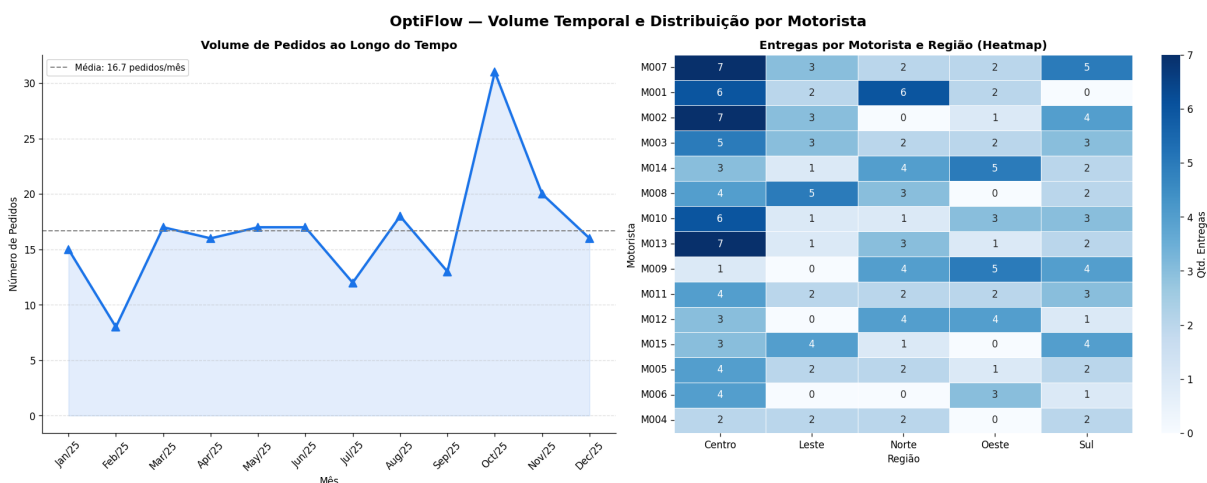


Figura 3 — Heatmap de pedidos por motorista e região (volume ao longo de 2025). Fonte: elaborado pelo autor (2025).

APÊNDICE D — SCRIPTS PYTHON: MODELOS DE OTIMIZAÇÃO

D.1 Problema 1 — Otimização de Rotas de Entrega ([problema_1_otimizacao_entregas/otimizacao.py](#))

```
import pulp, pandas as pd, numpy as np

# — Dados do problema —————
PEDIDOS = {
    "P001": {"peso_kg": 5.0, "regiao": "Centro"},
    "P002": {"peso_kg": 3.5, "regiao": "Norte"},
    "P003": {"peso_kg": 8.0, "regiao": "Sul"},
    "P004": {"peso_kg": 2.0, "regiao": "Leste"},
    "P005": {"peso_kg": 6.5, "regiao": "Oeste"},
    # ... P006 a P015 (15 pedidos no total)
}

ROTAS = {
    "R1": {"motorista": "M001", "disponivel": True, "custo_fixo": 25.0},
    "R2": {"motorista": "M003", "disponivel": True, "custo_fixo": 28.0},
    "R3": {"motorista": "M005", "disponivel": True, "custo_fixo": 22.0},
    "R4": {"motorista": "M007", "disponivel": False, "custo_fixo": 30.0}, #
    Indisponível
    "R5": {"motorista": "M009", "disponivel": True, "custo_fixo": 26.0},
}

PESO_MAXIMO_POR_ROTA = 20.0 # kg
DISTANCIA_MAXIMA = 60.0 # km
PARADAS_MAXIMAS = 5 # pedidos por rota

CUSTO_POR_KM = {"Centro": 1.20, "Norte": 1.50, "Sul": 1.40, "Leste": 1.30,
                "Oeste": 1.35}

# — Construção do modelo PLI (PuLP) —————
def construir_modelo():
    prob = pulp.LpProblem("OptiFlow_Otimizacao_Rotas", pulp.LpMinimize)

    # Variáveis: x[i,j] = 1 se pedido i vai para rota j; y[j] = 1 se rota j
    # ativada
    x = pulp.LpVariable.dicts("atribuir",
                              [(i, j) for i in nomes_pedidos for j in nomes_rotas], cat="Binary")
    y = pulp.LpVariable.dicts("ativar_rota", nomes_rotas, cat="Binary")

    # Função objetivo: minimizar custo fixo + custo variável
    prob += (pulp.lpSum(ROTAS[j]["custo_fixo"] * y[j] for j in nomes_rotas)
    +
             pulp.lpSum(custos.loc[i, j] * x[(i, j)]
```

```

        for i in nomes_pedidos for j in nomes_rotas))

# R1: Cada pedido atendido por exatamente uma rota
for i in nomes_pedidos:
    prob += pulp.lpSum(x[(i, j)] for j in nomes_rotas) == 1

# R2: Capacidade de peso por rota (peso_maximo * y[j])
for j in nomes_rotas:
    prob += pulp.lpSum(PEDIDOS[i]["peso_kg"] * x[(i, j)]
                        for i in nomes_pedidos) <= PESO_MAXIMO_POR_ROTA *
y[j]

# R3: Distância máxima por rota
for j in nomes_rotas:
    prob += pulp.lpSum(distancias.loc[i, j] * x[(i, j)]
                        for i in nomes_pedidos) <= DISTANCIA_MAXIMA *
y[j]

# R4: Máximo de paradas por rota
for j in nomes_rotas:
    prob += pulp.lpSum(x[(i, j)] for i in nomes_pedidos) <=
PARADAS_MAXIMAS * y[j]

# R5: Disponibilidade da rota
for j in nomes_rotas:
    prob += y[j] <= (1 if ROTAS[j]["disponivel"] else 0)

return prob, x, y

# — Execução —————
prob, x, y = construir_modelo()
prob.solve(pulp.PULP_CBC_CMD(msg=0))
print(f"Status: {pulp.LpStatus[prob.status]}")
print(f"Custo Total Mínimo: R$ {pulp.value(prob.objective):.2f}")

```

D.2 Problema 2 — Alocação de Motoristas por Região (problema_2_alocacao_motoristas/otimizacao.py)

```

import pulp, pandas as pd, numpy as np

# — Dados do problema —————
MOTORISTAS = {
    "M001": {"horas_disponiveis": 8.0, "nivel_experiencia": 3},
    "M002": {"horas_disponiveis": 6.0, "nivel_experiencia": 2},
    "M003": {"horas_disponiveis": 7.0, "nivel_experiencia": 3},
    "M004": {"horas_disponiveis": 5.0, "nivel_experiencia": 1},

```

```

    "M005": {"horas_disponiveis": 8.0, "nivel_experiencia": 2},
    # ... M006 a M010 (10 motoristas no total)
}

REGIOES = {
    "Centro": {"demanda_minima": 12, "max_motoristas": 3,
    "nivel_experiencia_min": 2},
    "Norte": {"demanda_minima": 10, "max_motoristas": 2,
    "nivel_experiencia_min": 1},
    "Sul": {"demanda_minima": 11, "max_motoristas": 2,
    "nivel_experiencia_min": 2},
    "Leste": {"demanda_minima": 9, "max_motoristas": 2,
    "nivel_experiencia_min": 1},
    "Oeste": {"demanda_minima": 10, "max_motoristas": 2,
    "nivel_experiencia_min": 1},
}

REGIOES_MAXIMAS_POR_MOTORISTA = 2

# eficiencia[m][r] = pedidos/hora (simulado, seed=7); horas_necessarias[m]
# [r] (seed=3)
np.random.seed(7)
eficiencia = pd.DataFrame(np.random.uniform(2.0, 5.0,
    size=(len(MOTORISTAS), len(REGIOES))),
    index=list(MOTORISTAS.keys()), columns=list(REGIOES.keys())).round(2)

# — Construção do modelo PLI (PuLP) —————
def construir_modelo():
    prob = pulp.LpProblem("OptiFlow_Alocacao_Motoristas", pulp.LpMaximize)

    # Variável: x[m,r] = 1 se motorista m alocado à região r
    x = pulp.LpVariable.dicts("alocar",
        [(m, r) for m in MOTORISTAS for r in REGIOES], cat="Binary")

    # Função objetivo: maximizar pedidos entregues totais
    prob += pulp.lpSum(
        eficiencia.loc[m, r] * MOTORISTAS[m]["horas_disponiveis"] * x[(m,
r)]
        for m in MOTORISTAS for r in REGIOES)

    # R1: Horas alocadas ≤ horas disponíveis
    for m in MOTORISTAS:
        prob += pulp.lpSum(horas_necessarias.loc[m, r] * x[(m, r)]
            for r in REGIOES) <= MOTORISTAS[m]
["horas_disponiveis"]

    # R2: Demanda mínima atendida por região
    for r in REGIOES:
        prob += pulp.lpSum(

```



```

        eficiencia.loc[m, r] * MOTORISTAS[m]["horas_disponiveis"] *
x[(m, r)]
        for m in MOTORISTAS) >= REGIOES[r]["demanda_minima"]

# R3: Máximo de motoristas por região
for r in REGIOES:
    prob += pulp.lpSum(x[(m, r)] for m in MOTORISTAS) <= REGIOES[r]
["max_motoristas"]

# R4: Nível mínimo de experiência exigido
for m in MOTORISTAS:
    for r in REGIOES:
        habilitado = 1 if MOTORISTAS[m]["nivel_experiencia"] >=
REGIOES[r]["nivel_experiencia_min"] else 0
        prob += x[(m, r)] <= habilitado

# R5: Máximo de regiões por motorista
for m in MOTORISTAS:
    prob += pulp.lpSum(x[(m, r)] for r in REGIOES) <=
REGIOES_MAXIMAS_POR_MOTORISTA

return prob, x

# — Execução —————
prob, x = construir_modelo()
prob.solve(pulp.PULP_CBC_CMD(msg=0))
print(f"Status: {pulp.LpStatus[prob.status]}")
print(f"Pedidos      estimados      (eficiência      total):
{pulp.value(prob.objective):.1f}")

```